

분산 AI 에이전트 크루 오케스트레이션 — 기술 백서

한 두뇌, 여러 손 · 두뇌는 분산, 결정은 사람

v1.0 · 2026 · DureClaw · [github.com/DureClaw](#) · 한국어판

초록

대부분의 공장·조직은 이미 **여러 플랫폼의 인프라**(엣지 디바이스, GPU 서버, Windows·macOS PC, 산업용 게이트웨이)를 잘 돌리고 있다. 문제는 **그 위에 AI를 얹는 비용**이다 — 모든 노드에 API 키를 심고, 데이터를 클라우드로 빼고, 통합을 새로 짜는 일은 무겁고 위험하다.

DureClaw는 갈아엎지 않는다. 이미 도는 인프라 위에 **하나의 실시간 협력 버스**를 얹고, 흩어진 머신을 **키 없이(keyless)** 한 줄로 합류시킨다. 추론은 **마스터 브레인**(Claude)이 흡수하고, 각 노드는 **서로 다른 손**(물리 액추에이터·브라우저·데스크톱 GUI)이 된다. 사람이 승인한 판단은 **결정론적 물로 컴파일**되어 다음부터는 **LLM 0회**로 재사용된다. 데이터 수집·분석에서 시작해 자동화와 **AX(AI Transformation)**로 가는 **시작점**이다.

1. 문제

흩어진 인프라에 AI를 도입할 때 반복되는 네 가지 벽:

벽	설명
키·비용	노드마다 모델 키를 심으면 키 관리·비용·레이트리미트가 폭증한다.
데이터 주권	원시 telemetry·영상·소음을 클라우드로 빼는 순간 규제·보안 리스크가 생긴다.
통합	이종 OS·CPU·도구(Pi·GPU·Win·Mac·PLC)를 잇는 일이 매년 일회성 통합 프로젝트가 된다.
신뢰	LLM이 직접 실행하면 비결정적이고 감사가 불가능하다.

핵심은 **데이터 확보 우선**이다 — 수집하지 못한 데이터는 영원히 **없는다**. 완벽한 파이프라인보다, 지금 도는 인프라에서 **가장 쉬운 방법으로 데이터를 모으기** 시작하는 것이 먼저다.

2. 핵심 아이디어

2.1 한 두뇌, 여러 손

마스터(Claude)가 **두뇌**, 각 노드는 **서로 다른 손**이다. 같은 버스·같은 keyless 위임으로, 하나의 추론이 **물리 세계·브라우저·데스크톱** 어디로든 손을 뻗는다.

```
└─ edgeclaw - 물리 세계의 손 (shell·sensor·GPIO·LED·부저·릴레이·신호탐)
마스터 두뇌 —fan-out—└─ webclaw - 브라우저의 손 (fetch·DOM, CORS-free)
(Claude·비전·추론)    └─ deskclaw - 데스크톱 GUI의 손 (스크린샷·클릭·타이핑·앱실행)
                        └─ adapters - 기존 도구의 손 (pico·nano·zero·null)
```

2.2 Keyless 엣지

엣지 노드는 **모델 키를 갖지 않는다**. 자연어 작업이 오면 노드는 프롬프트를 마스터 브레인의 `/brain/exec` 로 위임하고 결과만 받는다. 비용·키·레이트리밋은 마스터 한 곳이 흡수한다.

2.3 결정 동결 (LLM as compiler)

LLM을 매번 부르지 않는다. 사람이 **한 번 승인한 판단**은 결정론적 룰·매크로로 **동결(freeze)**되어, 같은 문제는 μ s 단위로 **LLM 0회** 재생된다. LLM은 *런타임 의존성*이 아니라 *컴파일러*다 — 이것이 닫힌 학습 루프의 핵심이다.

3. 아키텍처

3.1 협력 버스

DureClaw 버스는 **Phoenix Channel**(Elixir) 위의 WebSocket 한 채널이다. 와이어 프로토콜은 5-튜플 JSON 프레임이다:

```
[join_ref, ref, topic, event, payload]
```

- **합류**: `phx_join` → `work:<WORK_KEY>` 토픽. presence에 자기 역할·능력(capabilities)을 등록.
- **분배**: 마스터가 `task.assign` 을 fan-out (특정 노드 또는 `broadcast`).
- **회신**: 노드가 `task.result` 를 push (연결의 `join_ref` 포함 필수).
- **상태**: 승인·결정은 `task.result{approved}` 또는 `state.update` 로 전파.
- **하트비트**: `phoenix / heartbeat` 로 presence 유지.

REST 보조 API: `POST /api/task` , `GET /api/task-result/:id` , `/api/presence` , `/api/work-keys/latest` , `/api/health` . 인증은 Bearer 토큰(`0AH_SECRET`).

3.2 마스터 브레인

마스터는 노드들의 fan-in을 수집해 추론하고 **제안(propose)**만 한다 — 직접 실행은 0. keyless 위임 엔드포인트 (`/brain/exec`)로 엣지의 자연어 작업을 대신 처리한다. 이로써 *비전·추론*은 중앙, *행동*은 엣지로 분리된다.

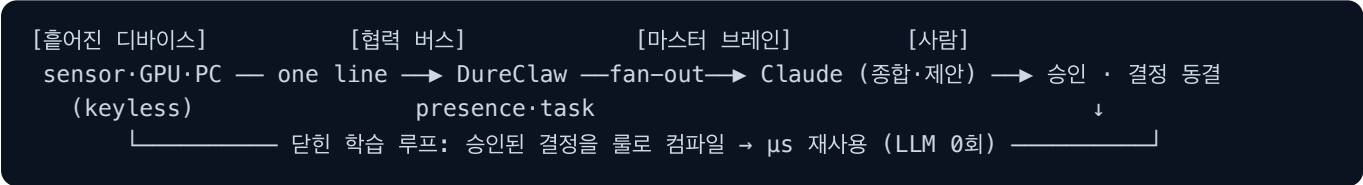
3.3 노드 패밀리

네이티브 노드 — 버스 우선 설계, 한 줄로 합류, keyless:

노드	손의 종류	구현
edgeclaw	물리 세계 — shell·sensor·GPIO·LED·부저·릴레이·신호탑·PA 음성	단일 정적 Go 바이너리, No-CGo. Win·Mac·Linux·Pi Zero(armv6)·riscv64… physical-edge는 gpiochip 노출 모든 Linux(Pi·Jetson·산업 게이트웨이)
webclaw	브라우저 — fetch·DOM (CORS-free, 상시)	Chrome MV3 확장, 순수 JS
deskclaw	데스크톱 GUI — 스크린샷·클릭·타이핑·키·앱실행 + RPA record→replay	Win/macOS, 순수 Go/No-CGo (OS 내장 도구)

어댑터 — 기존 오픈소스 도구에 `dureclaw/` 브리지를 더해 합류: `picoclaw`(Go) · `nanobot`(Py) · `zeroclaw`(Rust) · `nullclaw`(Zig). 같은 와이어 프로토콜, 같은 keyless 위임.

3.4 메시지 흐름



4. 결정 동결의 두 실재

추상 원칙이 아니라 실제 두 곳에서 동작한다:

- 1. **제조 라인 — 스킵 캐시.** 불량 조사 같은 판단을 1회차에 LLM이 수행하면, 그 결과를 결정론적 룰로 캐시한다. 같은 패턴은 이후 LLM 없이 즉시 적용된다. (Open MES Korea `EXT-5` 연동 허브.)
 - 2. **데스크톱 — RPA 매크로.** deskclaw에서 마스터(비전)가 프로그램 실행 → 지정 메뉴 클릭 → 입력을 한 단계씩 가르치면, deskclaw가 그 단계를 매크로(JSON)로 기록한다. 다음부터 `[RUN] <name>` 은 LLM 없이 결정론적으로 재생한다.
- 같은 닫힌 루프의 두 모습이다 — 한 번 가르치고, 이후로는 LLM 없이 재생한다.

5. 보안·거버넌스

- **사람이 결정한다.** 마스터는 제안만 하고, 중요한 변경은 사람이 승인한다. 승인 = 실행 트리거.
- **keyless 엿지.** 엿지에 모델 키가 없으므로 키 유출면이 줄어든다.
- **데이터는 엿지에.** 원시 telemetry·영상은 공장을 떠나지 않고, 정제된 컨텍스트만 마스터로 간다.
- **감사 가능.** 모든 쓰기·승인은 감사 로그로 남는다(Open MES Korea AuditLog). 결정 동결로 비결정성도 제거된다.
- **최소 인프라.** 브로커리스 HTTP/WS push 우선 — 새 메시지 큐·에이전트 런타임 없이 시작.

6. 적용 사례 — 분산 엣지 × 제조 MES

dure-factory 데모는 DureClaw 버스와 Open MES Korea(AI-native MES)를 결합한다. 골든 시나리오:

1. **센싱** — Pi(`executor@pi-cam`)·GPU·PC 노드가 키 없이 버스에 합류. 불량 트리거 발생.
2. **종합** — 마스터가 여러 노드의 신호를 fan-in 수집, 불량 원인을 역할별로 추론·제안.
3. **승인** — 품질관리자가 MES UI에서 검토·승인 → `quarantine` .
4. **물리 회신** — 승인이 버스로 broadcast되면 엣지 노드가  LED·부저·음성으로 발화. *AI 결정이 물리 세계로 되돌아온다.*
5. **동결** — 승인된 판단은 스킴 캐시 룰로 컴파일, 다음 동일 케이스는 LLM 0회.

불량 하나가 1초 만에 여러 머신으로 fan-out되고, Claude가 구조화된 제안을 종합하며, 사람이 승인하면 그 결정이 물리 세계로 되돌아오고, 재사용 가능한 룰로 동결된다 — 전 과정에 감사 추적이 남는다.

7. 시작점 → AX 여정

[기존 인프라] → ① 데이터 수집 → ② 분석(LLM) → ③ 자동화(룰 컴파일) → ④ AX
갈아엮지 않음 수집 분석 자동화 AI 전환

DureClaw는 ①에서 시작해 ④로 가는 *연속적인 길*을 제공한다. 각 단계는 이전 단계의 데이터·승인을 재사용하며, 닫힌 루프가 시간이 갈수록 LLM 의존도를 낮춘다.

8. 로드맵

- **deskclaw** — 접근성 트리 타게팅(좌표→요소, 창 이동에도 견고), 스크린샷→brain 업로드 루프, 매크로 파라미터.
- **edgeclaw** — 더 많은 액추에이터 프로파일(Modbus/OPC-UA 릴레이), 산업 프로토콜 SourceAdapter.
- **버스** — 결정 동결 룰의 버전 관리·배포, 멀티 워크키 오케스트레이션.
- **MES** — 시계열(EXT-1)·멀티미디어(EXT-2) 종합 조사 고도화, 예지보전(EXT-3).

9. 결론

DureClaw는 *새 AI 플랫폼*이 아니라, **이미 도는 인프라를 AI 크루로 바꾸는 얇은 층**이다. keyless 엣지, 한 두뇌-여러 손, 결정 동결 — 이 세 원칙으로 데이터 수집에서 AX까지 *갈아엮지 않고* 갈 수 있다.

데이터는 엣지 · 두뇌는 분산 · 학습은 닫힌 루프 · 결정은 사람.

부록 A. 와이어 프로토콜

이벤트	방향	의미
phx_join	node → bus	work:<WK> 합류 + presence 등록(역할·capabilities)
task.assign	bus → node	작업 분배 (to : 노드명 또는 broadcast)
task.result	node → bus	결과 회신 (반드시 연결의 join_ref 포함)
state.update	bus → node	상태·결정 전파 (예: decision_<lot>: quarantine)
heartbeat	node → bus	presence 유지 (15-30초)

부록 B. 노드 환경변수 (공통)

STATE_SERVER (버스 host:port) · OAH_SECRET (Bearer) · WORK_KEY ·
AGENT_NAME / AGENT_ROLE / CAPABILITIES · BRAIN_URL / BRAIN_TOKEN (keyless LLM 위임).